

Adam and Modern Optimizers

Presented by:

Badar, Caseja, Espiritu, Ibanez, Pangan,
Redondo, Soritcho

Why Fixed Learning Rates Are a Problem

- A single learning rate (η) is applied to all parameters
- But different parameters may require different update sizes
- Some parameters have large gradients $\rightarrow$ need small steps
- Others have small gradients $\rightarrow$ need bigger steps
- Fixed η cannot adapt $\rightarrow$ causes:
 - Slow convergence
 - Overshooting
 - Oscillations

Practical Guidelines: Adam vs SGD vs RMSprop

Optimizer	When to Use	Strength	Weakness
SGD	Simple / convex problems	Low memory, simple	Slow, noisy
RMSprop	Non-stationary problems	Stable updates	Needs tuning
Adam	Deep learning (default choice)	Fast + adaptive	More memory

AdaGrad

- **Adjusts learning rate for each parameter**
- **Uses accumulated squared gradients**
- **Formula idea:**
Step size $\downarrow$ as gradients accumulate
- **Automatically scales updates**
- **Works well for sparse data**

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch.12, §12.4.1, p. 259

AdaGrad

- Learning rate keeps shrinking
- Eventually becomes too small → stops learning

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch.12, §12.4.1, p. 259

AdaGrad

- Learning rate keeps shrinking
- Eventually becomes too small → stops learning

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch.12, §12.4.1, p. 259

RMSprop: Improved AdaGrad

- Uses moving (decaying) average instead of total sum
- Keeps learning rate stable over time
- Forget old gradients
- Focus on recent gradients

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch.12, §12.4.1, p. 259

Adam Optimizer – What It Is

- **Adaptive Moment Estimation**
- A popular optimization algorithm that combines two techniques: **Momentum** and **RMSprop**
- **1st Moment (Momentum)**: Tracks the running average of gradients — gives the optimizer a sense of direction/speed
- **2nd Moment (RMSprop)**: Tracks the running average of squared gradients — adapts the learning rate per parameter
- Result: **Fast convergence + stable updates**, even with noisy or sparse gradients

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch. 12, §12.4.3, p. 261

How Adam Works – The Formula

- Computes bias-corrected estimates for both moments to prevent early-training distortion
- Learning rate is automatically tuned per parameter — no single global rate
- Combines the best of both worlds:
- Momentum → accelerates learning in consistent gradient directions
- RMSprop → slows down learning for parameters with large gradient variance
- Hyperparameters: β_1 (momentum decay), β_2 (RMSprop decay), ε (numerical stability)

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch. 12, §12.4.3, p. 261

How Adam Works – The Formula

- Adam generally converges faster than SGD, Momentum, and RMSprop alone
- Performs well on:
- Noisy gradients
- Sparse data
- Deep/complex networks
- Trade-off: May generalize slightly worse than SGD in some tasks (e.g., image classification)
- Still the default choice in most modern deep learning workflows

Source: Kuznetsov, Intelligent Systems: From Theory to Applications, Ch. 12, §12.4.3, p. 263

Adam Update Equations

- **First Moment**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- **Second Moment**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- **Bias Correction**

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Adam Update Equations

- **Final Update Rule**

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

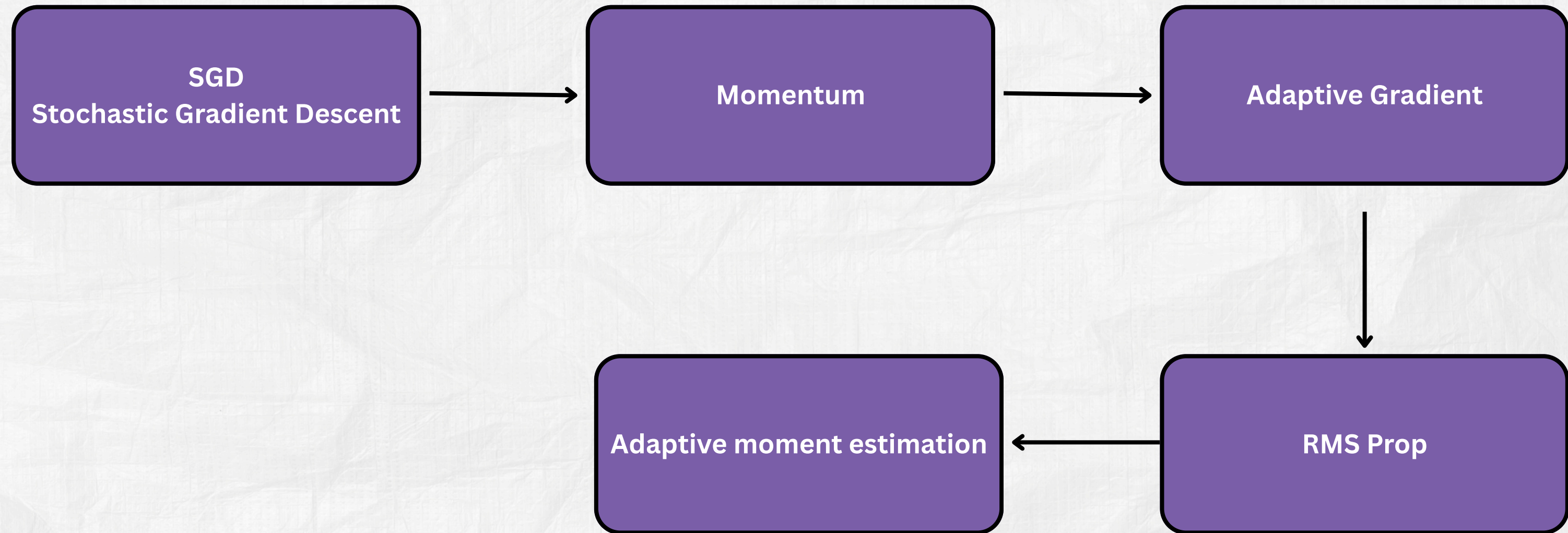
- **Standard Hyperparameters**

$$\beta_1 = 0.9$$

$$\beta_2 = 0.999$$

$$\epsilon = 10^{-8}$$

Algorithm Evolution



One full Adam step

Setup: single weight θ , step $t = 1$ (first ever update)

$\theta_0 = 0.5 \cdot g_1 = 0.3$ (current gradient) $\cdot \eta = 0.001$ (learning rate) $\cdot \beta_1 = 0.9 \cdot \beta_2 = 0.999 \cdot \epsilon = 1e-8$

Step 1 — first moment (momentum)

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\ &= 0.9 \times 0 + 0.1 \times 0.3 \\ m_1 &= 0.03 \end{aligned}$$

Exponential moving avg of gradient. Starts near zero at $t=1$ — hence bias correction is needed.

Step 2 — second moment (variance)

$$\begin{aligned} v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \\ &= 0.999 \times 0 + 0.001 \times 0.09 \\ v_1 &= 0.00009 \end{aligned}$$

Exponential moving avg of squared gradient. Even closer to zero than m_1 — $\beta_2 = 0.999$ is very conservative.

Step 3 — bias correction (mandatory)

$$\begin{aligned} \hat{m}_t &= m_t / (1 - \beta_1^t) \\ &= 0.03 / (1 - 0.9^1) = 0.03 / 0.1 \\ \hat{m}_1 &= 0.3 \end{aligned}$$

Without this, you'd use 0.03 — 10× too small. The correction inflates the biased estimate back to the true scale of the gradient. Bias correct on m .

Step 4 — bias correction on v (mandatory)

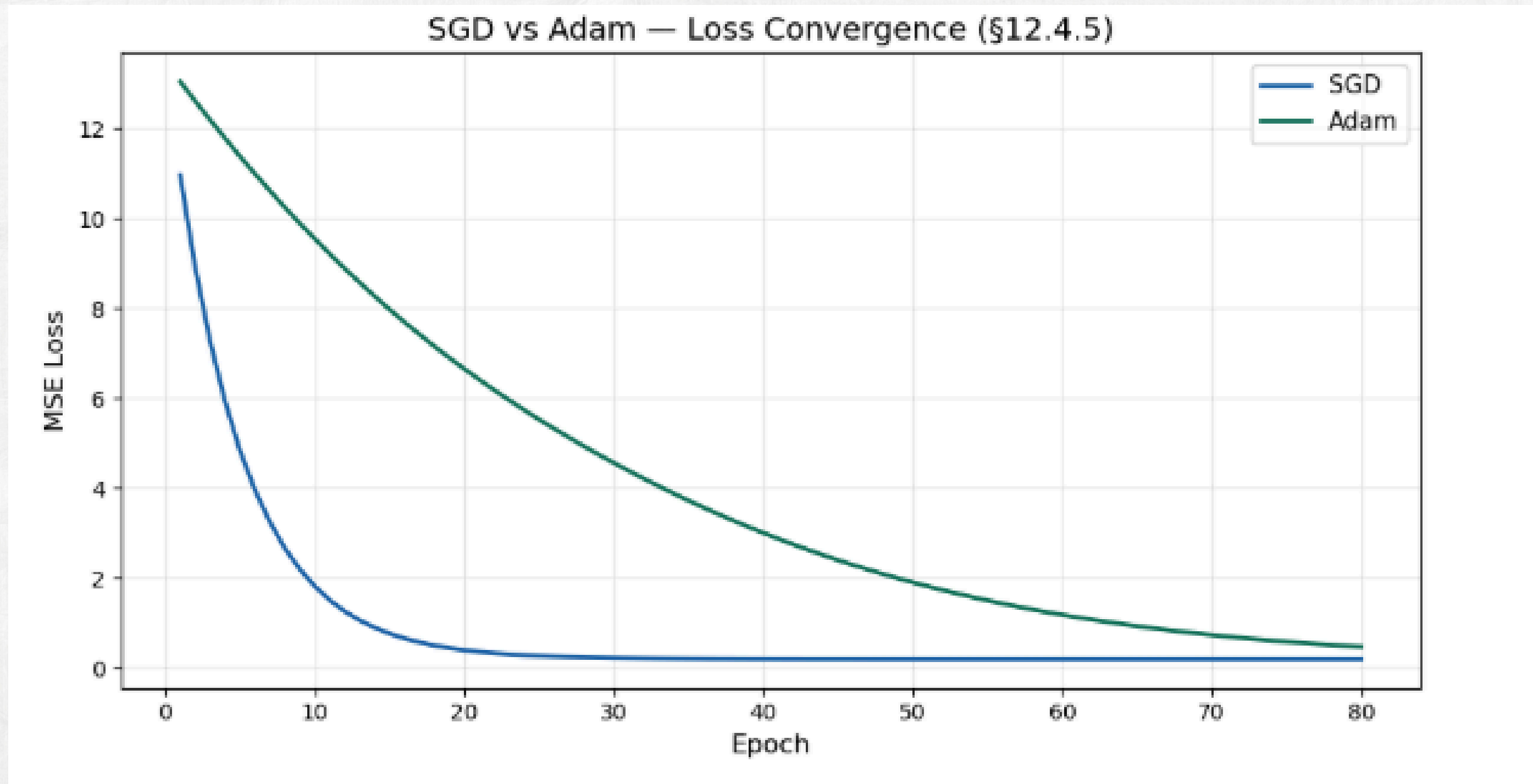
$$\begin{aligned} \hat{v}_t &= v_t / (1 - \beta_2^t) \\ &= 0.00009 / (1 - 0.999^1) = 0.00009 / 0.001 \\ \hat{v}_1 &= 0.09 \end{aligned}$$

Without correction, denominator = $\sqrt{0.00009} \approx 0.0095$ — making the first step ~31× too large. This is the catastrophic early divergence seen when bias correction is skipped.

Step 5 — parameter update

$$\begin{aligned} \theta_{t+1} &= \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \\ \text{denominator} &= \sqrt{0.09} + 1e-8 = 0.3 + 1e-8 \approx 0.3 \\ \text{step size} &= 0.001 \times 0.3 / 0.3 = 0.001 \\ \theta_1 &= 0.5 - 0.001 = 0.499 \end{aligned}$$

SGD vs Adam



Practical Guidelines: Adam vs SGD vs RMSprop

Thank You

